

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA



PROYECTO INDIVIDUAL

ESTUDIANTE:
CHOQUE TERRAZAS ELENA MAYOLI

DOCENTE: Ph.D. MOISES MARTIN SILVA CHOQUE
MATERIA: DESARROLLO DE APLICACIONES MULTIMEDIA

LA PAZ-BOLIVIA

2026

1. Introducción

El presente informe expone de manera detallada tres proyectos de desarrollo que, aunque distintos en alcance y propósito, comparten una misma base metodológica: la construcción de soluciones digitales orientadas al procesamiento de imágenes, la clasificación visual y la creación de experiencias multimedia interactivas. Los tres trabajos muestran una evolución clara en complejidad técnica y conceptual, pasando desde un filtro de suavizado básico sobre imágenes, hasta un sistema de clasificación de texturas con almacenamiento en base de datos y, finalmente, una producción multimedia en Unity con animación, audio, modelado 3D y exportación web.

La importancia de reunir estos proyectos en un solo informe radica en que permiten observar una progresión formativa muy útil para un perfil de ingeniería de software. En primer lugar, se aprecia la aplicación de algoritmos simples de análisis de píxeles y manipulación de imágenes. En segundo lugar, se observa el uso de estructuras de datos, comparación por distancia de color y persistencia en SQL Server. Finalmente, se alcanza un entorno de desarrollo más cercano a la producción multimedia, donde interactúan animaciones, modelado 3D, código en Unity, audio procesado externamente y despliegue en navegador.

Desde el punto de vista académico, este tipo de integración resulta valiosa porque no solo evidencia dominio de programación, sino también capacidad para resolver problemas diversos con herramientas adecuadas. En los tres casos, la solución elegida responde al contexto funcional requerido, lo que permite evaluar tanto la calidad técnica como la pertinencia de la implementación.

2. Objetivo del informe

El objetivo principal de este informe es describir, analizar y valorar los tres proyectos desarrollados, identificando su estructura, su comportamiento, las tecnologías utilizadas y el nivel de complejidad que presentan. Se busca además establecer una lectura comparativa entre ellos para evidenciar sus diferencias de enfoque y su aporte dentro de un proceso de aprendizaje técnico.

De forma específica, el informe pretende:

- Explicar el funcionamiento del filtro de suavizado 3x3.
- Analizar el sistema de clasificación de texturas y su relación con la base de datos.
- Describir la escena multimedia creada en Unity y su lógica de interacción.
- Identificar ventajas, limitaciones y oportunidades de mejora en cada proyecto.
- Presentar una visión integrada del conjunto como portafolio técnico.

3. Método de trabajo

La información de este informe se elaboró a partir del contenido proporcionado por la usuaria, que incluye fragmentos completos de código y descripciones de comportamiento de las aplicaciones. Se tomó cada proyecto por separado, se identificaron sus componentes principales y luego se analizaron en función de su objetivo, arquitectura y tecnologías involucradas. Para la redacción se siguió una estructura formal de informe con presentación, desarrollo, análisis y conclusión, tal como recomiendan las guías de redacción de informes académicos.

Además, se consideró que un informe de calidad debe redactarse de forma clara, objetiva y organizada, priorizando la comprensión del lector y evitando ambigüedades. Por ello, la exposición se dividió en secciones temáticas y se incluyó una comparación final que sintetiza las diferencias entre los tres proyectos.

4. Proyecto uno: filtro suavizado

Este proyecto crea una aplicación de escritorio en Windows Forms llamada “**Filtro Suavizado 3x3**”. Su función principal es cargar una imagen, aplicar un suavizado mediante una máscara 3x3 promediando los valores RGB de los píxeles vecinos, mostrar el resultado y permitir guardarlo. La interfaz incluye botones de cargar, suavizar y guardar, junto con dos PictureBox para comparar la imagen original y la filtrada.

Aspectos técnicos

- Lenguaje: **C#**.
- Interfaz: **Windows Forms**.
- Procesamiento: recorrido pixel a pixel con GetPixel y SetPixel.
- Filtro aplicado: promedio simple de vecindad 3x3.

Observaciones

- El algoritmo es fácil de entender y funcional para imágenes pequeñas o prácticas académicas.
- El uso de GetPixel/SetPixel puede volverlo lento en imágenes grandes.
- No maneja bordes con el mismo tratamiento que el centro, porque solo procesa desde 1 hasta Width - 1 y Height - 1.

El primer proyecto corresponde a una aplicación de escritorio desarrollada en C# con Windows Forms cuyo propósito es cargar una imagen, aplicar un filtro de suavizado 3x3 y guardar el resultado procesado. La interfaz es simple y funcional: contiene tres botones principales —cargar, suavizar y guardar— junto a dos PictureBox, uno para mostrar la imagen original y otro para visualizar la imagen filtrada.

La lógica del filtro consiste en recorrer la imagen píxel por píxel y, para cada posición interior, tomar la vecindad 3x3 alrededor del píxel actual. Luego se suman los valores RGB de los nueve píxeles, se calcula el promedio y se asigna ese color promedio al píxel correspondiente en la imagen resultado.

Esta operación tiene como efecto visual una reducción del ruido y una apariencia más suave en la imagen.

Desde el punto de vista algorítmico, se trata de una implementación clásica de filtrado por convolución simplificada, aunque realizada de manera manual mediante GetPixel y SetPixel. La elección de esta técnica es útil para fines académicos porque permite entender el concepto básico de procesamiento digital de imágenes sin depender de librerías avanzadas o abstracciones complejas.

Sin embargo, también presenta limitaciones. El uso de GetPixel y SetPixel suele ser lento en imágenes de gran tamaño, porque cada acceso implica una operación costosa a nivel de píxel. Además, el tratamiento de los bordes es incompleto, ya que el recorrido evita la primera y última fila y columna, dejando esos píxeles sin procesar. A pesar de ello, el proyecto cumple correctamente con su propósito y constituye una buena base para comprender el suavizado de imágenes.

En términos pedagógicos, este proyecto demuestra que la usuaria ha trabajado con conceptos esenciales como carga de archivos, control de eventos, manejo de bitmaps y manipulación básica de color. También evidencia que existe una comprensión práctica de la interfaz gráfica en Windows Forms y de cómo vincular interacción de usuario con procesamiento visual.

5. Proyecto dos: clasificador de texturas

Este proyecto es una aplicación más avanzada llamada “**Clasificador de Texturas - Fotogrametría**”. Carga una imagen, analiza la textura por bloques 3x3, compara cada promedio de color contra una lista de referencias y determina porcentajes de textura predominante entre **PASTO, TIERRA, CEMENTO y ASFALTO**. Además, genera una imagen sobrepuesta con colores representativos y guarda los resultados en una base de datos SQL Server local.

Funcionalidad principal

- Carga de imagen.
- Clasificación por similitud de color usando distancia euclidiana entre RGB.
- Cálculo de porcentajes por textura.
- Visualización de resultados en texto tipo consola dentro de la interfaz.
- Guardado en base de datos con imagen original en bytes.

Tecnologías usadas

- **C#**.
- **Windows Forms**.
- **Microsoft.Data.SqlClient** para SQL Server.
- Manejo de imágenes con **System.Drawing**.
- Persistencia en base de datos **TexturasDB** con tabla HistorialClasificaciones.

Estructura de datos destacable

La tabla incluye campos como nombre de imagen, fecha de clasificación, porcentajes por textura, textura predominante, confianza, ruta original e imagen en binario. Esto convierte al proyecto no solo en un clasificador visual, sino también en un sistema de historial de análisis.

Observaciones

- El enfoque es práctico para prototipos o tareas académicas.
- La clasificación está basada en colores de referencia, no en aprendizaje automático.
- La confianza se calcula como diferencia entre el porcentaje mayor y el segundo mayor, normalizada.

El segundo proyecto eleva significativamente el nivel de complejidad. Se trata de una aplicación llamada **Clasificador de Texturas - Fotogrametría**, también desarrollada en C# y Windows Forms, que analiza una imagen y determina qué tipo de textura predomina entre cuatro categorías: pasto, tierra, cemento y asfalto. Además, sobrepone colores representativos a la imagen clasificada y guarda los resultados en una base de datos SQL Server local.

El núcleo del proyecto se encuentra en el análisis por promedio de vecindad 3x3. Para cada píxel interior de la imagen, se calcula un color promedio de sus vecinos y luego se compara ese promedio con una lista de colores de referencia asociados a cada textura. La cercanía se determina mediante la distancia euclidiana en el espacio RGB. El algoritmo asigna cada zona de la imagen a la textura más cercana y contabiliza cuántas veces aparece cada clase.

A partir de esos conteos se calcula un porcentaje para cada textura. Posteriormente, el sistema identifica la textura predominante y genera una medida de confianza basada en la diferencia entre el porcentaje mayor y el segundo mayor. Esta lógica no corresponde a aprendizaje automático, pero sí a un sistema de clasificación heurística por similitud cromática, suficiente para escenarios controlados o imágenes con patrones de color relativamente estables.

La interfaz incluye dos PictureBox, uno para la imagen original y otro para la imagen sobrepuesta, además de botones para cargar, clasificar y guardar resultados. También incorpora una sección de resultados en texto monoespaciado, donde se muestran barras visuales que representan el porcentaje de cada textura. Esto aporta claridad al análisis y hace más comprensible la salida para el usuario.

Uno de los elementos más importantes del proyecto es la integración con base de datos. Se crea la base TexturasDB y la tabla HistorialClasificaciones, donde se almacenan nombre de imagen, fecha, porcentajes, textura predominante, confianza, ruta original e incluso la imagen en bytes. Esta capacidad de persistencia convierte al proyecto en algo más que un clasificador: también funciona como sistema de historial y trazabilidad de análisis.

Desde una perspectiva técnica, este proyecto demuestra el uso combinado de procesamiento de imágenes, estructura de datos, presentación de resultados y persistencia relacional. A diferencia del primer proyecto, aquí no solo se transforma una imagen, sino que se interpreta y se registra un resultado. El proyecto muestra una lógica de negocio más madura y una mejor articulación entre interfaz, cálculo y almacenamiento.

Entre sus fortalezas destacan la claridad de su propósito, la utilidad del sistema de clasificación y la integración con SQL Server. Entre sus limitaciones se encuentra el hecho de que la clasificación depende exclusivamente del color, por lo que puede fallar si la iluminación cambia o si las texturas reales no se parecen lo suficiente a los colores de referencia. Aun así, el proyecto sigue siendo sólido como propuesta académica y como base para una futura evolución hacia técnicas más avanzadas.

6. Proyecto tres: escena multimedia en Unity

El tercer proyecto ya no es una app de escritorio clásica, sino una **producción multimedia interactiva en Unity** con exportación web. La escena final incluye un suelo, un monigote 3D animado, una vaca 3D que se balancea con código, texto grande “LA VACA LOLA!!” al fondo y un audio de 15 segundos procesado previamente en Python.

Elementos de la escena

- **Suelo** con un Plane.
- **Monigote 3D** con Animator y animación en bucle.
- **Vaca 3D** importada desde archivos .obj, .mtl y .jpg.
- **Texto grande** con TextMeshPro.
- **Audio recortado y modificado** con pydub en Python.

Tecnologías y herramientas

- **Unity 6.**
- Exportación a **Web / WebGL.**
- **TextMeshPro** para el rótulo.
- **Pydub** y imageioffmpeg para el audio.
- Formato de modelo **OBJ + MTL + JPG** para la vaca.

Comportamiento

La propuesta busca sincronizar el audio con la animación, hacer que el monigote baile en loop y que la vaca rote/balancee en su eje, con orientación frontal y apoyada sobre el suelo. También se menciona la necesidad de exportar todo a HTML mediante la plataforma web de Unity.

El tercer proyecto marca un salto importante hacia el terreno de la producción multimedia interactiva. En este caso, la solución se desarrolla en **Unity 6** y está pensada para una escena única con exportación a web mediante la plataforma Web/WebGL. El objetivo no es clasificar ni filtrar

imágenes, sino construir una experiencia visual que combine animación, audio, modelado 3D y elementos decorativos de escena.

La composición descrita incluye varios elementos: un suelo tipo Plane, un monigote 3D con animación de baile, una vaca 3D que se balancea mediante código, un texto grande que dice “LA VACA LOLA!!” y un audio de 15 segundos previamente editado con Python usando pydub. Esto indica que la producción no se limita al motor Unity, sino que integra herramientas externas para preparar el material multimedia.

El monigote utiliza un Animator Controller con transiciones y estados de animación. Se mencionan transiciones automáticas con Has Exit Time, lo que permite que el personaje pase de un estado a otro de manera fluida. La vaca, por su parte, se mueve con un script que aplica rotación sinusoidal, haciendo que se incline o balancee de forma rítmica sobre un eje. Esta solución resulta apropiada cuando el modelo no cuenta con animaciones complejas, ya que permite simular movimiento con pocos recursos.

El aspecto de exportación también es relevante. Se describe la necesidad de compilar el proyecto en formato web, produciendo una carpeta con index.html, Build y TemplateData. Esa estructura es típica de despliegues web de Unity y permite ejecutar la aplicación en un navegador. Aunque el nombre exacto de la opción pueda variar dentro de la interfaz de Unity 6, la intención es clara: llevar la experiencia interactiva a un formato accesible desde la web.

Este proyecto es el más amplio de los tres, no solo por la cantidad de componentes involucrados, sino por la diversidad de tareas que exige. Incluye importación de modelos .obj, texturizado con .jpg, ajuste de materiales, control de rotación, sincronización con audio y preparación para exportación. A nivel de integración, representa una síntesis de habilidades que van desde el modelado y la animación hasta la programación y la distribución web.

7. Análisis comparativo

Proyecto	Objetivo	Interfaz	Procesamiento	Salida
----------	----------	----------	---------------	--------

Filtro Suavizado 3x3	Suavizar imagen	Windows Forms	Promedio 3x3 por píxel	Imagen filtrada
Clasificador de Texturas	Clasificar superficies por color	Windows Forms	Distancia RGB + porcentajes	Resultados + BD SQL
Producción Unity	Escena multimedia interactiva	Unity	Animación, rotación, audio, escena 3D	Build web/HTML

Los tres proyectos tienen una base común: todos nacen de una necesidad de representar, transformar o interpretar información visual. No obstante, su enfoque cambia con claridad a medida que aumenta la complejidad. El primero se concentra en el tratamiento matemático de la imagen. El segundo añade una capa de análisis semántico y almacenamiento. El tercero se aleja del análisis puro y se orienta a la experiencia interactiva y artística.

En el plano tecnológico, también hay diferencias notables. Los dos primeros se apoyan en C# y Windows Forms, mientras que el tercero usa Unity como motor principal y Python como herramienta auxiliar para preparar audio. Esto evidencia flexibilidad en el uso de tecnologías, algo muy valioso en ingeniería de software. Además, el salto de Windows Forms a Unity implica pasar de una interfaz convencional de escritorio a un entorno tridimensional con lógica de escena, animación y exportación multiplataforma.

En cuanto a la lógica interna, el primer proyecto se basa en un algoritmo directo de promedio. El segundo utiliza comparación con distancias y clasificación por referencia. El tercero recurre a animación y manipulación de transformaciones en tiempo real. Así, cada proyecto introduce un tipo distinto de razonamiento técnico: numérico, categórico y espacial.

Desde la perspectiva de dificultad, el primer proyecto es introductorio; el segundo es intermedio-avanzado; el tercero es integrador y multidisciplinario. Juntos, forman una secuencia coherente de aprendizaje que muestra cómo una base de programación puede evolucionar hacia soluciones más completas. En especial, el tercer proyecto demuestra capacidad para trabajar con múltiples recursos, algo esencial en entornos de desarrollo moderno.

8. Conclusión

En conjunto, los tres proyectos reflejan un proceso de formación técnica sólido y variado. El filtro suavizado demuestra dominio básico del procesamiento de imágenes. El clasificador de texturas introduce una solución más rica, con clasificación visual, sobreposición de resultados y persistencia en base de datos. La escena multimedia en Unity, por su parte, muestra integración de animación, audio, modelado y exportación web, lo que la convierte en la propuesta más ambiciosa del conjunto.

Desde una lectura académica, puede afirmarse que el valor del trabajo no reside solo en el resultado final de cada proyecto, sino también en la progresión que representan. Se observa una evolución desde el tratamiento de píxeles hasta la construcción de experiencias interactivas complejas. Esta transición es especialmente importante porque demuestra capacidad de adaptación, aprendizaje incremental y manejo de herramientas distintas en contextos diversos.

Asimismo, el conjunto evidencia competencias relevantes para ingeniería de software: programación orientada a eventos, manipulación de imágenes, uso de interfaces gráficas, clasificación heurística, integración con base de datos, preparación de activos multimedia y despliegue en la web. Todo ello sugiere un perfil técnico con potencial para abordar proyectos de carácter visual, interactivo y multimedia con bases sólidas.